Constructors & Destructors in Python

A Comprehensive Guide

Slide 1: Introduction

What are Constructors and Destructors?

- **Constructor**: Special method called when an object is created
- **Destructor**: Special method called when an object is destroyed
- Python provides `__init__()` for construction
- Python provides `__del__()` for destruction
- Both are part of Python's "magic methods" (dunder methods)

Slide 2: The `__init__` Method

Constructor in Python:

```
class Dog:
    def __init__(self):
        print("A Dog object is created!")
```

- Automatically called when an object is instantiated
- Its return type is `None`
- First parameter is always `self` (reference to current instance)

Slide 3: Creating Objects - The Process

What happens when you write `Dog()`?

1. Python allocates memory for the new object
2. Python calls `__new__()` to create the raw object
3. Python calls `__init__()` to initialize the object
4. The object is returned to the caller

```
# This single line triggers all steps above
my_dog = Dog()
```

Slide 4: `__new__` vs `__init__`

Two-step construction:

```
class Example:
    def __new__(cls):
        print("1. __new__ called")
        return super().__new__(cls)

    def __init__(self):
        print("2. __init__ called")

obj = Example()
# Output:
# 1. __new__ called
# 2. __init__ called
```

- `__new__()` creates the object (rarely overridden)
- `__init__()` initializes the object (commonly overridden)

Slide 5: Constructors with Parameters

Passing arguments to `__init__`:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Create objects with different values
s1 = Student("Alice", 20)
s2 = Student("Bob", 22)

print(s1.name)  # Alice
print(s2.name)  # Bob
```

Slide 6: Default Parameters in Constructors

Providing default values:

```
class Student:
    def __init__(self, name="Unknown", age=18):
        self.name = name
        self.age = age

s1 = Student("Alice", 20)  # name=Alice, age=20
s2 = Student("Bob")        # name=Bob, age=18
s3 = Student()              # name=Unknown, age=18
```

- Default parameters make some arguments optional
- Must be defined at the end of the parameter list

Slide 7: Multiple Constructors in Python

Python doesn't support multiple `__init__` methods directly:

```
class Example:
    # This overwrites the first __init__
    def __init__(self, a):
        self.a = a

    # Only this one exists at runtime
    def __init__(self, a, b):
        self.a = a
        self.b = b
```

Workaround: Use default parameters or `@classmethod`:

```
class Student:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, data):
        name, age = data.split(",")
        return cls(name, int(age))
```

Slide 8: Factory Methods as Alternative Constructors

Using `@classmethod` for flexible object creation:

```
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day

    @classmethod
    def from_string(cls, date_str):
        parts = date_str.split("-")
        return cls(int(parts[0]), int(parts[1]), int(parts[2]))

    @classmethod
    def today(cls):
```

```
        from datetime import datetime
        now = datetime.now()
        return cls(now.year, now.month, now.day)

d1 = Date(2024, 1, 15)
d2 = Date.from_string("2024-01-15")
d3 = Date.today()
```

Slide 9: Instance Attributes in Constructor

Setting up object state:

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        self.transactions = []  # Empty list for tracking

    def deposit(self, amount):
        self.balance += amount
        self.transactions.append(f"Deposit: ${amount}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            self.transactions.append(f"Withdraw: ${amount}")
```

Slide 10: The `__del__` Method

Destructor in Python:

```
class Resource:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} created")

    def __del__(self):
        print(f"{self.name} destroyed")
```

- Called when an object is about to be destroyed
- Its primary use: cleanup (close files, release network connections, etc.)
- Not guaranteed to be called immediately

Slide 11: When Does `__del__` Get Called?

Object destruction scenarios:

```
class Demo:
    def __init__(self, id):
        self.id = id
        print(f"Object {self.id} created")

    def __del__(self):
        print(f"Object {self.id} destroyed")

# 1. Object goes out of scope
def test():
    d = Demo(1)  # Created
    # d is destroyed when function ends

# 2. Reference count drops to zero
d2 = Demo(2)
d3 = d2          # Reference count = 2
del d2           # Reference count = 1 (not destroyed yet)
del d3           # Reference count = 0 (destroyed now)

# 3. Program ends
d4 = Demo(4)     # Destroyed when program exits
```

Slide 12: Reference Counting & `__del__`

Reference counting mechanism:

```
class Demo:
    def __del__(self):
        print("Destroyed")

d1 = Demo()
d2 = d1    # ref count = 2
d3 = d1    # ref count = 3
del d1     # ref count = 2 (no __del__ called)
del d2     # ref count = 1 (no __del__ called)
del d3     # ref count = 0 (__del__ called now)
```

- Python uses reference counting for memory management
- `__del__` runs when reference count reaches zero
- Circular references can prevent `__del__` from running

Slide 13: Circular References Problem

Objects referencing each other:

```python
class Node:
    def __init__(self, name):
        self.name = name
        self.ref = None
        print(f"{name} created")

    def __del__(self):
        print(f"{name} destroyed")

# Circular reference
a = Node("A")
b = Node("B")
a.ref = b
b.ref = a

del a
del b
# __del__ may NOT be called!
# Circular references prevent cleanup
```

- Python's garbage collector handles circular references
- But `__del__` timing becomes unpredictable
- Better to use context managers for cleanup

Slide 14: Context Managers as Alternative

The `with` statement pattern:

```python
class FileHandler:
    def __init__(self, filename):
        self.filename = filename

    def __enter__(self):
        self.file = open(self.filename, 'r')
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        print(f"File {self.filename} closed")

# Usage
with FileHandler("data.txt") as f:
    content = f.read()
# File automatically closed, no need for __del__
```

- `__enter__` / `__exit__` are more predictable than `__del__`
- Preferred for resource management in Python

Slide 15: Constructor Initialization Patterns

Pattern 1: Basic initialization:

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y
```

Pattern 2: Validation in constructor:

```
class Person:
    def __init__(self, name, age):
        if not name:
            raise ValueError("Name cannot be empty")
        if age < 0:
            raise ValueError("Age cannot be negative")
        self.name = name
        self.age = age
```

Slide 16: Inheritance and Constructors

Calling parent constructor:

```
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"Animal: {name} created")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call parent constructor
        self.breed = breed
        print(f"Dog: {name} ({breed}) created")

# Output:
# Animal: Buddy created
# Dog: Buddy (Golden Retriever) created
buddy = Dog("Buddy", "Golden Retriever")
```

Slide 17: MRO and Constructors in Multiple Inheritance

Method Resolution Order:

```python
class A:
    def __init__(self):
        print("A.__init__")
        super().__init__()

class B:
    def __init__(self):
        print("B.__init__")
        super().__init__()

class C(A, B):
    def __init__(self):
        print("C.__init__")
        super().__init__()

c = C()
# Output:
# C.__init__
# A.__init__
# B.__init__
```

- `super()` follows MRO (Method Resolution Order)
- Ensures all parent constructors are called
- Linearization algorithm (C3) determines order

Slide 18: Attribute Inheritance in Constructor

Accessing parent attributes:

```python
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)
        self.model = model

    def info(self):
        return f"{self.brand} {self.model}"

car = Car("Toyota", "Camry")
print(car.info())  # Toyota Camry
print(car.brand)   # Toyota (inherited)
print(car.model)   # Camry (own)
```

Slide 19: Preventing Object Creation

Making a class non-instantiable:

```
class Constants:
    def __init__(self):
        raise TypeError("Cannot instantiate Constants class")

    PI = 3.14159
    E = 2.71828

# constants = Constants()  # TypeError!

print(Constants.PI)  # 3.14159
print(Constants.E)   # 2.71828
```

Using ABC (Abstract Base Classes):

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

# shape = Shape()  # TypeError!
```

Slide 20: Singleton Pattern with Constructor

Controlling object creation:

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        # Only runs once
        if not hasattr(self, 'initialized'):
            self.initialized = True
            print("Singleton initialized")

s1 = Singleton()
s2 = Singleton()
print(s1 is s2)  # True (same object)
```

- `__new__` controls object creation
- `__init__` runs on every call (need to guard)
- Useful for shared resources (database connections, config)

Slide 21: Property-Based Constructor Validation

Using properties for controlled attribute setting:

```
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Temperature below absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

t = Temperature(25)
print(t.fahrenheit)  # 77.0
```

Slide 22: Decorator for Constructor Logging

Automatically logging object creation:

```
def log_construction(cls):
    original_init = cls.__init__

    def new_init(self, *args, **kwargs):
        print(f"Creating {cls.__name__} object")
        original_init(self, *args, **kwargs)
        print(f"{cls.__name__} object created: {self}")

    cls.__init__ = new_init
    return cls

@log_construction
class Person:
    def __init__(self, name):
        self.name = name
```

```
    def __repr__(self):
        return f"Person({self.name})"

p = Person("Alice")
# Output:
# Creating Person object
# Person object created: Person(Alice)
```

Slide 23: `__init_subclass__` Hook

Reacting to subclass creation:

```
class PluginBase:
    plugins = {}

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        cls.plugins[cls.__name__] = cls
        print(f"Plugin registered: {cls.__name__}")

class AudioPlugin(PluginBase):
    def __init__(self):
        print("AudioPlugin constructed")

class VideoPlugin(PluginBase):
    def __init__(self):
        print("VideoPlugin constructed")

# Output:
# Plugin registered: AudioPlugin
# Plugin registered: VideoPlugin
print(PluginBase.plugins)
# {'AudioPlugin': <class '__main__.AudioPlugin'>, ...}
```

Slide 24: Weak References and Destructors

Using weakref to avoid circular reference issues:

```
import weakref

class Resource:
    def __del__(self):
        print("Resource cleaned up")

class Container:
    def __init__(self, resource):
```

```
        self.resource_ref = weakref.ref(resource)

    def get_resource(self):
        return self.resource_ref()

r = Resource()
c = Container(r)
del r  # __del__ called immediately (no circular ref)
```

- `weakref.ref()` doesn't increase reference count
- Prevents circular reference memory leaks
- Enables predictable `__del__` behavior

Slide 25: `__del__` Caveats

Important warnings:

```
class Fragile:
    def __del__(self):
        # BAD: Accessing global/module state
        print("Destroyed")  # OK
        self.file.close()   # DANGEROUS

    def __del__(self):
        # BAD: Raising exceptions
        raise Exception("Error in destructor!")
```

Rules for `__del__`:

- Don't rely on it for critical cleanup
- Don't raise exceptions (they're ignored)
- Don't access global variables (may be cleaned up already)
- Use context managers or `atexit` instead

Slide 26: `__del__` vs Context Managers

Aspect	`__del__`	Context Manager
When called	Unpredictable	Deterministic
Exception handling	Exceptions ignored	Exceptions propagate
Circular references	Problematic	Works fine
Resource cleanup	Not guaranteed	Guaranteed
Pythonic	Less preferred	Preferred

```
# Preferred: Context Manager
with open("file.txt") as f:
    data = f.read()

# Avoid: Relying on __del__
class FileReader:
    def __init__(self, path):
        self.file = open(path)
    def __del__(self):
        self.file.close()  # Not guaranteed!
```

Slide 27: Garbage Collection in Python

Python's GC system:

```
import gc

class Demo:
    def __del__(self):
        print("Cleaned up by GC")

# Manual GC control
gc.collect()  # Force garbage collection
print(gc.get_threshold())  # (700, 10, 10)
```

- Python uses reference counting + generational GC
- Generation 0: New objects (checked frequently)
- Generation 1: Survivors from Gen 0
- Generation 2: Survivors from Gen 1
- GC handles circular references that ref counting misses

Slide 28: Complete Example - Database Connection

```
class DatabaseConnection:
    _instances = {}

    def __new__(cls, db_name):
        if db_name not in cls._instances:
            instance = super().__new__(cls)
            instance._initialized = False
            cls._instances[db_name] = instance
        return cls._instances[db_name]

    def __init__(self, db_name):
        if not self._initialized:
```

```
            self.db_name = db_name
            self.connected = False
            self._initialized = True

    def connect(self):
        print(f"Connecting to {self.db_name}...")
        self.connected = True

    def disconnect(self):
        print(f"Disconnecting from {self.db_name}...")
        self.connected = True

    def __enter__(self):
        self.connect()
        return self

    def __exit__(self, *args):
        self.disconnect()

# Usage
with DatabaseConnection("users") as db:
    print(f"Connected: {db.connected}")
```

Slide 29: Constructor & Destructor Best Practices

Do's:

- ✓ Initialize all attributes in `__init__`
- ✓ Use default parameters for flexibility
- ✓ Validate inputs in constructor
- ✓ Use `@classmethod` for alternative constructors
- ✓ Use context managers for resource cleanup

Don'ts:

- ✗ Rely on `__del__` for critical operations
- ✗ Raise exceptions in `__del__`
- ✗ Create circular references with `__del__`
- ✗ Access global state in `__del__`

Slide 30: Summary

Constructors (`__init__`):

- Called when object is created
- Initializes object state
- Can take parameters with defaults
- `@classmethod` provides alternative constructors

Destructors (`__del__`):

- Called before object is destroyed
- Used for cleanup (but unreliable)
- Context managers are preferred alternative

Key Takeaway:

- Python emphasizes explicit, predictable patterns
- `__init__` is essential, `__del__` is optional
- "Explicit is better than implicit" - Zen of Python

Slide 31: Resources & Further Reading

Official Documentation:

- Python `__init__`: docs.python.org/3/reference/datamodel.html
- Python `__del__`: docs.python.org/3/reference/datamodel.html
- Python GC: docs.python.org/3/library/gc.html

Topics for Further Study:

- `__new__` and metaclasses
- Descriptor protocol
- `__slots__` for memory optimization
- `atexit` module for cleanup
- `weakref` module
- `contextlib` utilities

Thank You!

Questions?